

Parcialito IV : Concurrencia, Transacciones y Recuperación

Concurrencia Importante: Usar la tabla pedida para la resolución de los ejercicios I. Para las siguientes planificaciones:

A. Dibujar los grafos de precedencia

B. Cantidad y Listado los conflictos

C. Determinar cuáles son serializables (justificando correctamente en una oración).

D. Es recuperable? (justificando correctamente en una oración).

E. En caso de no recuperable, que tendríamos que modificar para que sea recuperable? (justificando correctamente en una oración).

1. bT1; bT2; bT3; RT1(X); WT2(X); RT3(X); WT1(Y); RT2(Y); WT3(Z); cT1; cT2; cT3;

1.1	
Grafo de precedencia	<pre> graph LR t1((t1)) -- "x,y" --> t2((t2)) t2 -- "x" --> t3((t3)) </pre>
Cantidad de conflictos	3
Listado de conflictos	RT1(X) -> WT2(X); WT2(X) -> RT3(X); WT1(Y) -> RT2(Y);
¿Es serializable? ¿Por qué?	Si porque como el grafo no es acíclico, existe manera de serializar las transacciones.
¿Es recuperable? ¿Por qué?	Sí, porque en todas las situaciones donde una transacción lee un valor escrito por otra, la transacción que escribió primero realiza commit antes de la que lo leyó.
Modificación para hacerlo recuperable	-

2. bT1; bT2; bT3; RT1(X); WT1(X); RT2(X); WT2(X); RT2(Y); WT2(Y); RT3(Y); WT3(Y); cT3;
cT2; cT1;

1.2	
Grafo de precedencia	<pre> graph LR t1((t1)) -- x --> t2((t2)) t2 -- y --> t3((t3)) </pre>
Cantidad de conflictos	6
Listado de conflictos	RT1(X) -> WT2(X); WT1(X) -> RT2(X); WT1(X) -> WT2(X); RT2(Y) -> WT3(Y); WT2(Y) -> WT3(Y); WT2(Y) -> RT3(Y);
¿Es serializable? ¿Por qué?	Si porque como el grafo no es acíclico, existe manera de serializar las transacciones.
¿Es recuperable? ¿Por qué?	No porque existe alguna transacción que lee un valor escrito por otra y la que lee realiza COMMIT antes que la que lo modifica. Por ejemplo: WT1(X) -> RT2(X); T2 commitea después de T1.
Modificación para hacerlo recuperable	Cambiando el orden de commits Antes: cT3; cT2; cT1 Cambiado: cT1; cT2; cT3

2. Un SGBD implementa el algoritmo de recuperación UNDO/REDO con checkpoint activo. Luego de una falla, el sistema encuentra el siguiente archivo de log (a la derecha) incompleto:

Complete los valores faltantes y explique cómo se llevará a cabo el procedimiento de recuperación, indicando hasta qué punto del archivo de log se deberá retroceder (justificando por qué), y qué cambios deberán ser realizados en disco y en el archivo de log. Para la entrega, entregue el archivo de log completo, y en lo posible, ponga las partes faltantes en color ROJO.

Nro línea | Log
01 (BEGIN, T1);
02 (WRITE T1, A, 1, __);
03 (BEGIN CKPT, __);
04 (COMMIT, T1);
05 (BEGIN, T2);
06 (WRITE T2, A, 2, 4);
07 (COMMIT, T2);
08 (END CKPT);
09 (BEGIN, T3);
10 (BEGIN CKPT, __);
11 (WRITE T3, A, __ , 3);
12 (WRITE T3, B, 6, 7);

Resolución

Nro línea | Log
01 (BEGIN, T1);
02 (WRITE T1, A, 1, **2**);
03 (BEGIN CKPT, **T1**);
04 (COMMIT, T1);
05 (BEGIN, T2);
06 (WRITE T2, A, 2, 4);
07 (COMMIT, T2);
08 (END CKPT);
09 (BEGIN, T3);
10 (BEGIN CKPT, **T3**);
11 (WRITE T3, A, **4**, 3);
12 (WRITE T3, B, 6, 7);

Procedimiento de recuperación:

El BEGIN CKPT de la línea 10 está descartado porque no está su END CKPT. Por lo que pasó al siguiente checkpoint completo.

El checkpoint completo es de la línea 3 hasta la línea 8;

Se rehacen las transacciones que están después del BEGIN CKPT porque todo lo anterior ya está asegurado que esta en disco (y ninguna se deshace porque tanto T1 y T2 commitearon). En este caso, se rehace (REDO) la línea 6 (A = 4). (En REDO no hace falta

hacer los cambios anteriores del BEGIN CKPT porque ya se encuentran en disco. Si se hacen no estaría mal porque pisa con el mismo valor, pero es innecesario).

Por último, T3 nunca comitea por lo que sus transacciones deberían deshacerse. En este caso se deshace la línea 12 (quedando B=6) y luego la línea 11 (quedando A=4). Además, ya que no comitea T3 debería hacer un ABORT. (En UNDO se lee todo el archivo de abajo hacia arriba para ver las transacciones incompletas; sin commit).

Nro línea | Log

```
01 (BEGIN, T1);
02 (WRITE T1, A, 1, 2 );
03 (BEGIN CKPT, T1);
04 (COMMIT, T1);
05 (BEGIN, T2);
06 (WRITE T2, A, 2, 4);
07 (COMMIT, T2);
08 (END CKPT);
09 (BEGIN, T3);
10 (BEGIN CKPT, T3);
11 (WRITE T3, A, 4, 3);
12 (WRITE T3, B, 6, 7);
13 (ABORT, T3);
```